

ZK-KYA: A Privacy-Preserving Identity and Authorization Layer for AI Agents

Aaditya Pai

Yegan Dhaivakumar
Rujing Li

Jie Wang

Columbia University

COMS 4995-001: Science of Blockchain, Spring 2026

Abstract

As AI agents move from research prototypes into deployed systems that interact with financial services, healthcare providers, and enterprise APIs, a practical authorization gap has emerged. Existing delegation mechanisms such as OAuth tokens and API keys depend on centralized policy servers, expose sensitive permission details in plaintext, and are not designed for autonomous agents acting without a user online. This paper presents ZK-KYA (Zero-Knowledge Know Your Agent), a proof-of-concept system that addresses this gap by combining ECDSA credential signing, an on-chain Poseidon hash commitment registry in Solidity, and Groth16 zero-knowledge proofs generated with circom and snarkjs. An agent proves it holds a valid, unexpired, in-scope credential without revealing any credential fields. The system enforces spending caps, service scope, and expiry as circuit constraints rather than policy rules. We implement and evaluate a full end-to-end prototype covering two demo scenarios (financial payment and healthcare record access), two rejection modes (exceeded limit and expired credential), and a Groth16 versus PLONK benchmark. Proof generation takes approximately 450ms, on-chain verification consumes roughly 318k gas, and the circuit uses 890 R1CS constraints.

1 Introduction and Motivation

AI agents are being deployed in settings where they must take real actions with real consequences. A user who delegates an agent to pay a utility bill needs the bank to verify that the agent is authorized for that specific transaction at that specific amount, without the user being online at the time. A healthcare platform receiving a record request from an automated system needs to confirm the agent has appropriate access, without that access being tied to a static API key that could be stolen or shared.

Current approaches to this problem are not well suited for autonomous agents. OAuth 2.0 requires a user to be present for the authorization grant and assumes the client application is an interactive service, not an agent acting asynchronously. API keys are static secrets with no scoped permissions, no expiry enforcement, and no cryptographic binding to a specific action or user. Both approaches depend on a central authorization server that must be trusted, online, and honest. If it is compromised or unavailable, authorization fails or becomes insecure.

Blockchain infrastructure offers a different model. A smart contract can serve as a tamper-resistant permission registry with no trusted operator. Credentials can be committed on-chain in hashed form, so their existence is publicly verifiable without their contents being visible. Zero-knowledge

proofs allow an agent to prove it holds a valid authorization without revealing the credential itself. Once the proof and commitment are on-chain, enforcement does not depend on any single party.

This project builds that model as a working prototype. The result is a system where a user issues a scoped credential offline, its hash is registered on-chain via a verified transaction, and the agent later proves the authorization using a zk-SNARK. The smart contract verifies the proof and the commitment in a single view call. The credential fields never appear on-chain.

2 Background

2.1 ECDSA Signatures and Ethereum Identity

Credential issuance in ZK-KYA is authenticated using ECDSA over secp256k1, the signature scheme used for Ethereum transaction authorization. A user signs a structured credential using their private key, producing a signature that can be verified by anyone who knows the corresponding public key or Ethereum address. The smart contract uses the native `ecrecover` precompile to recover the signer address and confirm it matches the claimed issuer.

This approach follows the account-based identity model used by Ethereum, where control of a private key defines identity. In contrast, decentralized identity frameworks such as those built on Verifiable Credentials and decentralized identifiers introduce additional abstraction layers for identity management. Our design instead leverages Ethereum-native primitives directly, reducing system complexity while remaining compatible with existing wallet infrastructure.

2.2 Smart Contracts and On-Chain State

The credential registry is implemented as a Solidity contract deployed on a local Hardhat node. It maintains a mapping from commitment hashes to booleans, recording which credentials have been validly issued. The `issueCredential` function writes to this mapping after verifying the issuer’s signature, while `verifyProof` reads from it before delegating verification to the zk-SNARK verifier contract.

This design uses Ethereum’s account-based state model to provide tamper-resistant, publicly verifiable storage. Compared to systems such as Indicio or Dock, which often rely on off-chain registries or application-layer verification, the on-chain registry ensures that credential validity can be checked independently by any participant without trusting a centralized service.

2.3 Hash Commitments

Rather than storing credential contents on-chain, the registry stores a Poseidon hash of the credential fields. Poseidon is a hash function optimized for zero-knowledge circuits, enabling efficient evaluation inside arithmetic constraint systems. The commitment binds all credential attributes to a single field element without revealing any underlying data.

Commitment-based designs are widely used in privacy-preserving systems, including zk-SNARK-based protocols such as Zcash. However, while such systems typically use commitments to hide transaction details or enable selective disclosure, ZK-KYA uses commitments as the anchor for later authorization proofs, where constraints over hidden attributes are enforced directly in zero knowledge.

2.4 zk-SNARKs, Groth16, and circom

To authorize an action, an agent must prove it holds a credential satisfying specific constraints. We use Groth16, a pairing-based zk-SNARK that produces constant-size proofs and supports efficient on-chain verification using the bn128 pairing precompile. The circuit is written in circom and compiled to an R1CS representation. The proving key is generated via a phase-2 trusted setup using snarkjs, and the corresponding verifier contract is deployed alongside the registry.

Groth16 is widely used in blockchain applications due to its low verification cost, as seen in systems such as Zcash. Alternative proof systems such as PLONK avoid per-circuit trusted setup but incur higher proof size and verification overhead. In our setting, Groth16 provides a practical trade-off between efficiency and setup complexity for on-chain authorization.

2.5 Gas Costs and On-Chain Verification

Ethereum charges gas for each computational step executed on-chain. Pairing operations on bn128 are particularly expensive, making zk-SNARK verification the dominant cost in the system. In our measurements, on-chain Groth16 verification consumes approximately 318k gas, while credential issuance, consisting of a signature verification via `ecrecover` and a single storage write, costs roughly 57k gas.

These costs are consistent with other zk-SNARK-based applications deployed on Ethereum and represent a key consideration for real-world deployment. While acceptable for high-value transactions, they motivate further optimizations such as batching, aggregation, or off-chain verification strategies.

3 System Design

3.1 Overview

The system has five components: a Solidity credential registry contract, a Groth16 verifier contract generated by snarkjs, a JavaScript identity and signing module, a circom ZK circuit, and a Node.js backend with a React frontend serving as the mock service and demo interface.

The authorization flow has two phases. In the issuance phase, a user signs a credential struct, the backend computes its Poseidon commitment, and a transaction calls `issueCredential` on the registry. In the authorization phase, the backend builds a circuit witness, generates a Groth16 proof, and calls `verifyProof` on the registry, which checks the commitment and delegates proof checking to the verifier contract.

3.2 Credential Registry Contract

`CredentialRegistry.sol` is the core contract. It holds a reference to the deployed `Groth16Verifier` address and maintains a `mapping(uint256 => bool)` called `registeredCommitments`.

The `issueCredential` function accepts all seven credential fields plus the commitment and the ECDSA signature. It reconstructs the message hash from the fields using `keccak256(abi.encode(...))` and calls `ecrecover` internally to recover the signer. If the recovered address matches the claimed issuer and the commitment is not already registered, it writes `true` to the mapping and emits a `CredentialIssued` event.

The `verifyProof` function takes a Groth16 proof in the standard snarkjs calldata format (pA, pB, pC, pubSignals) and a five-element public signals array. It first checks that the commitment in `pubSignals[0]` is registered, then calls the verifier contract. The verifier performs the Groth16 pairing check and returns true if and only if the proof is valid for the given public inputs.

3.3 Groth16 Verifier Contract

`AuthorizationVerifier.sol` is generated by `snarkjs zkey export solidityverifier` from the final proving key. It contains the verification key hardcoded as Solidity constants (alpha, beta, gamma, delta, and the IC points) and performs the full pairing check in EVM assembly using the bn128 precompiles at addresses 6, 7, and 8. This contract must be regenerated whenever the circuit trusted setup is rerun.

3.4 Identity and Signing Module

Credential signing is handled by three JavaScript files in the `scripts/` directory. `credentialSchema.cjs` defines the credential struct and enumerates service and action constants (BANK=1, HEALTH=2, PAY_BILL=1, READ_RECORD=2). `signCredential.cjs` ABI-encodes the seven credential fields, applies `keccak256`, and calls `wallet.signMessage` from ethers.js v6, which prepends the Ethereum signed message prefix before signing. `hashCredential.cjs` uses `circomlibjs.buildPoseidon()` to compute the same Poseidon hash that the circuit will later verify.

3.5 ZK Circuit

`credentialAuthorization.circom` is the core cryptographic component. It has twelve input signals: seven private credential fields (issuer, agent, service, action, maxAmount, expiry, nonce) plus one private commitment input, and four public inputs (requestedService, requestedAction, requestedAmount, currentTime). The circuit proves three things:

1. **Commitment integrity:** $\text{Poseidon}(\text{issuer}, \text{agent}, \text{service}, \text{action}, \text{maxAmount}, \text{expiry}, \text{nonce}) = \text{commitment}$. This binds the private credential fields to the on-chain commitment without revealing them.
2. **Expiry enforcement:** $\text{currentTime} \leq \text{expiry}$, checked using `LessEqThan(252)`.
3. **Scope enforcement:** $\text{requestedService} = \text{service}$, $\text{requestedAction} = \text{action}$, and $\text{requestedAmount} \leq \text{maxAmount}$, checked with equality constraints and `LessEqThan(252)`.

The circuit compiles to 890 non-linear R1CS constraints and 705 linear constraints, for 1,601 wires total.

3.6 Why ECDSA Stays Outside the ZK Circuit

A natural question is why the ECDSA signature is not verified inside the circuit, which would make the proof self-contained without requiring the on-chain `ecrecover` call. The reason is practical: ECDSA verification over secp256k1 requires non-native field arithmetic inside the circom circuit, because secp256k1 operates over a different prime field than the bn128 field used by Groth16. Implementing this in circom balloons the constraint count into the hundreds of thousands and makes proof generation prohibitively slow. The design separates concerns cleanly: `ecrecover` handles signature verification at issuance time, and the ZK circuit handles scope and expiry enforcement

at authorization time. Once a commitment is on-chain, the contract has already confirmed it was legitimately issued.

3.7 Backend and Frontend

The Node.js backend (`backend/routes/api.cjs`) exposes four POST endpoints. `/api/issue-credential` signs and submits a credential on-chain. `/api/authorize-payment` builds the circuit input, runs witness generation and proof generation using the compiled wasm and snarkjs CLI, and submits the proof to the registry's `verifyProof` view function. `/api/demo-expired` issues an already-expired credential and attempts to prove it, demonstrating circuit-level expiry enforcement. `/api/benchmark-plonk` runs the same input through both Groth16 and PLONK and returns a side-by-side timing and size comparison.

The React frontend provides a step-by-step interface for issuing credentials, authorizing requests in finance and healthcare modes, triggering the rejection demos, and viewing a session audit log and live metrics panel.

4 Implementation Details

4.1 Credential Representation

A credential is a seven-field struct: issuer address, agent address, service ID (integer), action ID (integer), maxAmount (integer), expiry (Unix timestamp), and nonce. The nonce distinguishes otherwise identical credentials and prevents two credentials with the same parameters from collapsing to the same commitment.

All address fields are converted to field elements using `BigInt(address.toLowerCase())` before being passed to Poseidon. This conversion is consistent between the off-chain `hashCredential.cjs` and the circuit, ensuring the commitment computed off-chain matches what the circuit re-derives from the private inputs.

4.2 Commitment Generation

The Poseidon hash is computed over the seven credential fields in order: `Poseidon(issuer, agent, service, action, maxAmount, expiry, nonce)`. The same computation is performed off-chain using `circomlibjs.buildPoseidon()` when preparing the issuance payload, and inside the circuit using the `Poseidon(7)` template from `circomlib`. The equality constraint `commitment === h.out` in the circuit ensures that the public commitment input matches the hash derived from the private credential fields.

4.3 Proof Generation and Verification

When the agent wants to act, the backend constructs a JSON input file with all twelve circuit signals. It then runs witness generation using the compiled wasm:

```
node generate_witness.js credentialAuthorization.wasm input.json witness
.wtns
```

If any circuit constraint is violated (for example, requestedAmount exceeds maxAmount), witness generation throws `Assert Failed` and no proof is produced. This is how the backend distinguishes constraint violations from server errors.

Proof generation runs:

```
snarkjs groth16 prove credentialAuthorization_final.zkey witness.wtns
proof.json public.json
```

The resulting proof is converted to the Solidity calldata format (pA as uint[2], pB as uint[2][2] with coordinate swap for the G2 point, pC as uint[2]) and passed to `CredentialRegistry.verifyProof`.

4.4 Trusted Setup

The Groth16 trusted setup is performed locally by `scripts/buildCircuit.sh` using `snarkjs`. The script generates a fresh powers-of-tau file using `snarkjs powersoftau new bn128 12`, applies a single deterministic contribution, runs `powersoftau prepare phase2`, performs the Groth16 phase-2 setup, and applies one contribution with fixed entropy. For a demo and proof-of-concept, this single-party setup is acceptable. A production system would require a multi-party ceremony with independent, publicly verifiable contributions.

4.5 Simplifications and Assumptions

Several simplifications distinguish this from a production system. The private key used for issuance is a hardcoded Hardhat test key, not a real user wallet. There is no credential revocation mechanism. The nonce provides replay resistance at the commitment level but there is no global nullifier set to prevent proof replay. The circuit uses 252-bit comparators, which are correct for the field size but have not been formally audited. All evaluation numbers come from a local Hardhat node, so gas costs reflect EVM simulation rather than mainnet conditions.

5 Demo Scenarios

5.1 Successful Financial Authorization

The user issues a finance credential with `service=1` (BANK), `action=1` (PAY_BILL), `maxAmount=100`, and an expiry one hour in the future. The commitment is registered on-chain in about 57k gas. The agent then requests authorization for a \$50 payment. The backend generates a valid witness, produces a Groth16 proof in roughly 450ms, and calls `verifyProof` on-chain. The proof verifies and the registry returns true.

5.2 Successful Healthcare Authorization

The same flow with `service=2` (HEALTH), `action=2` (READ_RECORD), and `maxAmount=0`. The circuit's scope constraints confirm that the requesting agent has the correct service and action in its credential. The authorization succeeds.

5.3 Exceeded Spending Limit

The user issues a credential with `maxAmount=100`. The agent requests \$200. Witness generation fails because the `LessEqThan(252)` constraint for `requestedAmount` is not satisfiable. The backend catches the `Assert Failed` error and returns `reason: "constraint_violation"`. No proof is generated and no chain call is made.

5.4 Expired Credential

The demo issues a credential with expiry set one hour in the past. The backend then attempts authorization using the current timestamp as `currentTime`. Witness generation fails because `currentTime > expiry` violates the `LessEqThan(252)` constraint. The response returns `reason: "expired_credential"`.

5.5 Cases Not Shown

The scope mismatch case (agent holding a BANK credential attempting to authorize a HEALTH action) is enforced by the equality constraints `requestedService === service` and `requestedAction === action` but is not explicitly demoed as a dedicated button. An invalid credential (commitment not registered on-chain) is caught at the `registeredCommitments` check before the proof is even evaluated.

6 Evaluation

Table 1 summarizes the measured prototype metrics. All measurements were taken on a local Hardhat node running on a MacBook Pro (Apple M-series, Node.js 25.9.0).

Metric	Value	Notes
Issuance gas	56,903	includes ecrecover + storage write
Groth16 proof time	≈ 450 ms	snarkjs CLI on wasm witness
PLONK proof time	$\approx 2,360$ ms	$\approx 5\times$ slower than Groth16
Groth16 proof size	≈ 800 bytes	JSON encoding
PLONK proof size	$\approx 2,250$ bytes	$\approx 2.8\times$ larger than Groth16
On-chain verify call	≈ 16 ms	eth.call latency, local node
Verification gas (est.)	$\approx 318,000$	estimateGas on verifyProof
Witness generation	≈ 100 ms	wasm witness generator
R1CS constraints (non-linear)	890	from circom compile output
R1CS constraints (linear)	705	
Wires	1,601	

Table 1: Measured prototype metrics.

Proof size comparison. Raw Groth16 proofs consist of two G1 points and one G2 point on bn128, totaling 256 bytes of compressed curve data. In the JSON encoding used here the size is approximately 800 bytes. PLONK proofs are larger because they include more polynomial commitments; our measurements show approximately 2,250 bytes, consistent with published benchmarks for PLONK on comparable constraint counts.

Gas cost discussion. The 318k gas estimate for `verifyProof` reflects the cost of three bn128 precompile calls (ecAdd at address 6, ecMul at address 7, and ecPairing at address 8) plus the commitment lookup. For reference, a standard ERC-20 transfer costs roughly 21k gas. On-chain Groth16 verification is expensive but bounded and predictable, which is a key property for production deployment. With a gas price of 20 gwei and ETH at \$3,000, a single verification would cost approximately \$0.02 on mainnet, which is acceptable for high-value agent transactions.

Trusted setup. The local setup runs in roughly 3 minutes on the same machine. A production setup would require a multi-party ceremony, which does not affect proving or verification times but is a prerequisite for security.

7 Limitations and Future Work

Trusted setup. The current Groth16 setup uses a single locally-generated contribution, which is appropriate for a prototype but not production-secure. Anyone who knows the toxic waste from the ceremony could forge proofs. A real deployment would require a public multi-party ceremony with independent contributors. PLONK eliminates the per-circuit ceremony but still requires a universal structured reference string, which our implementation generates locally.

Credential revocation. Once a commitment is registered on-chain, there is no mechanism to revoke it before its expiry. A production system would need either a revocation registry (an on-chain list of invalidated commitments) or a shorter expiry window combined with a renewal flow.

Proof replay. The current system does not maintain a nullifier set, so a valid proof for a given commitment could in principle be replayed. Adding a nullifier (a hash of the proof's public inputs plus a service-specific session nonce) committed on-chain after each successful authorization would prevent this.

Gas costs. The 318k gas verification cost makes per-request on-chain verification expensive for low-value transactions. Batch verification or a validity rollup approach, where many proofs are aggregated into one on-chain verification, would reduce per-transaction costs significantly.

More expressive credential formats. The current credential schema is intentionally narrow, scoped to a single service, a single action type, and a flat spending cap, which is sufficient for the financial payment and healthcare demos. An interesting direction for future work would be generalizing this into an application-agnostic credential format that could support a broader class of agent actions: file system access, API rate-limited operations, multi-step workflows, or cross-service delegation chains. One approach worth exploring would be representing permitted actions as a Merkle tree of allowed operation identifiers, where the circuit checks that a requested action is a member of the tree without revealing the full permission set. This would keep the on-chain footprint and proof size constant regardless of how rich the credential's permissions are, and would move ZK-KYA closer to a general-purpose agent authorization primitive rather than a demo-scoped prototype.

Key management. The prototype uses a hardcoded Hardhat test private key. A production system would require a hardware wallet or HSM for the issuer key, and the agent key would need its own management lifecycle including rotation and revocation.

Frontend and service integration. The current mock service API does not enforce anything beyond returning a boolean from the proof check. A real service would need to integrate the on-chain verification result into its access control decision and handle edge cases such as network failures during the proof check.

8 Conclusion

ZK-KYA demonstrates a working end-to-end system for privacy-preserving agent authorization. It shows that the combination of ECDSA-signed credentials, on-chain Poseidon hash commitments, and Groth16 zero-knowledge proofs can enforce scoped, expiring permissions without ever exposing the underlying credential contents. The circuit enforces spending caps, service scope, and expiry as hard mathematical constraints rather than policy rules, which means an agent provably cannot exceed its permissions, not because a server refuses it, but because no valid proof can be generated.

The prototype covers both a financial payment scenario and a healthcare record access scenario, demonstrates two distinct rejection modes, and includes a Groth16 versus PLONK benchmark. The numbers are practical: 450ms to prove, 318k gas to verify, 890 constraints. These are well within range for high-value agent transactions where the cost of a cryptographic check is small relative to the value of the action being authorized.

The broader point is that blockchain infrastructure and zero-knowledge proofs are a natural fit for the agent authorization problem. Once a credential is issued and committed on-chain, its enforcement is independent of any operator. No policy server needs to be trusted, no API key needs to be kept secret, and no centralized service needs to be online. The rules are in the circuit and the contract, and they cannot be changed without redeployment. As autonomous agents become more capable and more widely deployed, this kind of cryptographic enforcement is likely to become important infrastructure.

Team Contributions

All four team members contributed equally to this project, each accounting for approximately 25% of the total work. The table below outlines the primary responsibilities of each member.

Member	Primary Contributions	Share
Aaditya Pai	ZK circuit design and implementation (<code>credentialAuthorization.circom</code>), trusted setup ceremony, circuit constraint validation, and Groth16 vs. PLONK benchmark design	25%
Jie Wang	Node.js backend API development (<code>api.cjs</code>), credential signing module (<code>signCredential.cjs</code>), witness generation pipeline, and proof size analysis	25%
Yegan Dhaivakumar	Solidity smart contract development (<code>CredentialRegistry.sol</code> , Groth16 verifier integration), deployment scripting, on-chain commitment registry design, and credential hashing module (<code>hashCredential.cjs</code>)	25%
Rujing Li	Demo scenario design and edge case testing (expired credential, exceeded spending limit, scope mismatch), evaluation and benchmarking (gas measurements, PLONK timing comparison)	25%

Table 2: *
Team contribution breakdown.

Code Repository

The full implementation is available at: <https://github.com/aaditya79/ZK-KYA-Authorization-Layer>

Video Demonstration

A walkthrough of the full end-to-end prototype is available at: <https://youtu.be/Qdqfr4XhGxo>

The video covers credential issuance, financial and healthcare authorization, the exceeded spending limit and expired credential rejection demos, and the Groth16 vs. PLONK benchmark.

AI Assistance Disclosure

During the development of this project, Claude (Anthropic) was used as a coding assistant for selected implementation tasks. Specifically, AI assistance was used to help scaffold the React frontend UI components and styling, assist with debugging backend API integration issues between the Node.js server and the Hardhat node, and provide guidance on deployment scripting. The core technical contributions of this project, including the circom circuit design, the Solidity smart contracts, the ZK proof pipeline, the cryptographic credential scheme, and the overall system architecture, were designed and implemented by the team. AI tools were used in a supporting capacity comparable to consulting documentation or a coding reference.